



前端性能优化详解

1. 课程代码



optimization-demo.zip

89.90MB



2. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)

- [📄 阿里/字节面试高频真题解析与9月考点预测](#)
- [📄 阿里&字节9月面试高频真题预测](#)

3. 前端为什么要进行性能优化？

性能优化不是可选项，而是现代Web开发的必备技能。其主要原因可以归结为以下四点：

1. 用户体验

- 留存率与转化率：加载速度每延迟1秒，可能导致转化率下降7%，页面浏览量减少11%，用户满意度降低16%。一个快速响应的网站能显著提升用户留存、转化和复购。
- 减少跳出率：用户耐心有限，如果页面加载时间过长（尤其是超过3秒），超过一半的用户会选择离开。
- 流畅的交互：优化不仅关乎加载，也关乎运行时。避免卡顿、掉帧的交互，能提供丝滑的用户体验。

2. 搜索引擎优化

- 排名权重：谷歌、百度等主流搜索引擎已将页面加载速度作为重要的搜索排名因素。更快的网站意味着更好的SEO和更多的自然流量。
- 核心Web指标：Google提出的LCP（最大内容绘制）、FID（首次输入延迟）、CLS（累积布局偏移）等核心指标直接与搜索排名挂钩。

3. 业务目标与收入

- 直接关联：如上所述，更好的用户体验和SEO会带来更多的流量和更高的转化，最终直接影响收入和利润。例如，亚马逊曾统计，页面加载每慢100毫秒，销售额就会减少1%。

4. 硬件与网络环境的多样性

- 移动端优先：在移动网络（3G/4G/5G）和性能相对较弱的移动设备上，性能优化尤为重要，能确保所有用户都能获得可用的体验。
- 全球化：对于全球用户，需要考虑不同地区网络延迟的差异，优化资源加载策略。

4. 常见的性能优化包含哪些

定量指标：

1. Largest Contentful Paint : 最大内容绘制，测量加载性能。理想值 < 2.5 秒。
2. First Input Delay : 首次输入延迟，测量交互性。理想值 < 100 毫秒。
3. Cumulative Layout Shift : 累积布局偏移，测量视觉稳定性。理想值 < 0.1。

4.1.1 编译构建时优化

1. 体积优化：terser 压缩、混淆

2. 资源优化

- 图片：webp等、srcset 基于dpi尺寸
 - 字体：预加载关键字体: `<link rel="preload" as="font">`。
 - 懒加载：script defer。
 - CDN
- ##### 3. 减少 HTTP 请求
- 文件合并: 在 HTTP/1.1 时代常用，但在 HTTP/2+ 下需权衡（多路复用减少了合并的必要性，合并可能影响缓存粒度）。小图标合并成雪碧图仍有价值。
 - 内联关键 CSS/JS: 首屏内容 & 骨架屏 所必需的小段 CSS/JS 直接内联在 HTML 中，避免阻塞渲染的请求。

4.1.2 运行时优化

1. 优化 CSS

- a. 避免深层嵌套选择器: 降低匹配复杂度。
- b. 减少过度使用昂贵的属性: 如 `box-shadow` , `border-radius` , `filter`

2. 优化 JavaScript 执行

- a. 避免长任务: 将耗时 JS 任务分解成小块 (使用 `setTimeout` , `requestAnimationFrame` , `requestIdleCallback` , Web Workers)。
- b. 防抖和节流: 控制高频事件 (resize, scroll, input) 的处理频率。
- c. Web Workers: 将复杂的计算任务移出主线程，避免阻塞 UI 渲染和交互响应。

4. 利用浏览器缓存

- **强缓存 (Cache-Control, Expires):** 为静态资源设置较长的过期时间 (e.g., `max-age=31536000`, `immutable`)。
- **协商缓存 (ETag, Last-Modified):** 确保用户代理能有效验证资源是否过期。
- **Service Worker:** 实现更精细的缓存策略 (网络优先、缓存优先等)，支持离线访问。PWA 的核心。

5. 减少资源体积

- Code Splitting
- Tree Shaking
- Webpack Bundle Analyzer。

6. 预加载、预连接、预获取

- `<link rel="preload">` : ** 强制浏览器提前加载关键资源 (字体、首屏图片、关键 JS/CSS)。
- `<link rel="preconnect">` : ** 提前建立与关键第三方源 (CDN, API 域名, 分析服务) 的连接 (DNS 查找、TCP 握手、TLS 协商)。
- `<link rel="prefetch">` : ** 在浏览器空闲时预取用户下一步可能需要的资源 (如下一页的内容、非首屏图片)。
- `<link rel="dns-prefetch">` : ** 提前解析域名 DNS (比 `preconnect` 更轻量)。

7. HTTP/2 或 HTTP/3

- 确保服务器和 CDN 支持 HTTP/2 (多路复用、头部压缩、服务器推送 - 需谨慎使用) 或更先进的 HTTP/3 (基于 QUIC, 改进弱网性能)。

- d. 优化事件处理: 使用事件委托, 及时移除不再需要的事件监听器。
- e. 虚拟列表: 对于超长列表, 仅渲染可视区域内的项 (`react-window`, `vue-virtual-scroller`)。

3. 优化渲染流程:

- a. 减少重排和重绘: 集中修改 DOM 样式 (使用 `class` 切换), 避免逐个修改。离线操作 DOM (使用 `DocumentFragment`, `cloneNode`, 隐藏元素修改完再显示)。
- b. GPU 加速: 对动画元素使用 `transform` 和 `opacity` 属性, 它们通常在合成层处理, 避免触发重排重绘。

4. 框架优化: 避免不必要的渲染

- a. React: 使用 `React.memo`, `PureComponent`, `useMemo`, `useCallback`。优化 Context 的使用。
- b. Vue: 利用 `v-once`, `v-memo` (Vue 3), 计算属性缓存, 合理设计组件。

4.1.3 用户体验

1. 骨架屏
2. 占位符
3. 进度条
4. 交互优化 Hover click etc
5. 资源缓存 LRU

5. 如何平稳收获offer?



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



5.1 专业一对一

近期一对一

- 目 xxx-2.5年-React-24K
- 目 xxx-1年-React-15k
- 目 xxx-7年-Vue-20k

报名学员专享

目 13. 如何写出高质量的简历（For 付费学员）

5.2 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

6. 如何基于已有项目进行性能优化

6.1 源文件体积分析

使用 rollup-plugin-visualizer 分析 npm 包构建产物。

另外，分析当前项目中哪些文件的体积过大

Code block

```
1  #!/usr/bin/env node
2
3  const fs = require('fs');
4  const path = require('path');
5
6  /**
7   * 格式化文件大小显示
8   * @param {number} bytes - 字节数
9   * @returns {string} 格式化后的大小字符串
10  */
11 function formatFileSize(bytes) {
12   if (bytes === 0) return '0 Bytes';
13
14   const k = 1024;
15   const sizes = ['Bytes', 'KB', 'MB', 'GB'];
16   const i = Math.floor(Math.log(bytes) / Math.log(k));
17
18   return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
19 }
20
21 /**
22  * 递归遍历目录，收集文件信息
23  * @param {string} dir - 目录路径
24  * @param {Array} files - 收集文件信息的数组
25  */
26 function traverseDirectory(dir, files) {
27   const entries = fs.readdirSync(dir, { withFileTypes: true });
28
29   for (const entry of entries) {
30     const fullPath = path.join(dir, entry.name);
31
32     if (entry.isDirectory()) {
33       // 递归遍历子目录
34       traverseDirectory(fullPath, files);
35     } else if (entry.isFile()) {
36       // 获取文件信息
37       const stats = fs.statSync(fullPath);
38       files.push({
39         path: fullPath,
40         size: stats.size,
41         formattedSize: formatFileSize(stats.size)
42       });
43     }
44   }
45 }
46
47 /**
48  * 分析项目src目录下的文件体积
49  */
50 function analyzeFileSizes() {
51   const projectRoot = path.join(__dirname, '..');
52   const srcDir = path.join(projectRoot, 'src');
53   const files = [];
54
55   console.log('分析目录:', srcDir);
56   console.log('正在收集文件信息...');
57
58   try {
```

```
59 // 遍历src目录
60 traverseDirectory(srcDir, files);
61
62 // 按文件大小降序排序
63 files.sort((a, b) => b.size - a.size);
64
65 // 计算总大小
66 const totalSize = files.reduce((sum, file) => sum + file.size, 0);
67
68 console.log('\n=== 文件大小分析结果 ===');
69 console.log(`总文件数: ${files.length}`);
70 console.log(`总大小: ${formatFileSize(totalSize)}`);
71 console.log(`\n体积最大的文件 (前20个):`);
72 console.log('-----');
73 console.log('序号 | 相对路径 | 大小 | 占比');
74 console.log('-----');
75
76 // 显示前20个最大的文件
77 const topFiles = files.slice(0, 20);
78 topFiles.forEach((file, index) => {
79     const relativePath = path.relative(projectRoot, file.path);
80     const percentage = ((file.size / totalSize) * 100).toFixed(2);
81     console.log(`${String(index + 1).padStart(3)} | ${relativePath.padEnd(50)} |
${file.formattedSize.padEnd(10)} | ${percentage}%`);
82 });
83
84 console.log('-----');
85
86 // 按文件类型统计
87 console.log('\n按文件类型统计:');
88 const typeStats = {};
89
90 files.forEach(file => {
91     const ext = path.extname(file.path).toLowerCase() || '.noext';
92     if (!typeStats[ext]) {
93         typeStats[ext] = { count: 0, size: 0 };
94     }
95     typeStats[ext].count++;
96     typeStats[ext].size += file.size;
97 });
98
99 // 转换为数组并排序
100 const typeStatsArray = Object.entries(typeStats)
101     .map(([type, data]) => ({
102         type,
103         count: data.count,
104         size: data.size,
105         formattedSize: formatFileSize(data.size),
106         percentage: ((data.size / totalSize) * 100).toFixed(2)
107     }))
108     .sort((a, b) => b.size - a.size);
109
110 console.log('文件类型 | 文件数 | 总大小 | 占比');
111 console.log('-----');
112
113 typeStatsArray.forEach(stat => {
114     console.log(`${stat.type.padEnd(10)} | ${String(stat.count).padStart(6)} |
${stat.formattedSize.padEnd(10)} | ${stat.percentage}%`);
115 });
116
117 } catch (error) {
118     console.error('分析文件大小时出错:', error.message);
119 }
```

```
119     process.exit(1);
120   }
121 }
122
123 // 执行分析
124 analyzeFileSizes();
```

序号	相对路径	大小	占比
1	src\assets\images\characters\character_6.png	7.55 MB	24.08%
2	src\assets\images\characters\character_4.png	6.12 MB	19.50%
3	src\assets\images\characters\character_3.png	3.46 MB	11.04%
4	src\assets\images\characters\character_2.png	3.35 MB	10.67%
5	src\assets\images\characters\character_5.png	3.11 MB	9.92%
6	src\assets\images\cover\cover_3.jpg	2.95 MB	9.40%
7	src\assets\images\characters\character_1.png	2.94 MB	9.37%
8	src\assets\images\cover\cover_4.jpg	755.79 KB	2.35%
9	src\assets\images\avatars\avatar-4.png	61.33 KB	0.19%
10	src\assets\images\avatars\avatar-6.png	52.45 KB	0.16%
11	src\assets\images\background\banner-1.png	49.74 KB	0.15%
12	src\assets\images\avatars\avatar-7.png	47.36 KB	0.15%
13	src\assets\images\avatars\avatar-1.png	43.56 KB	0.14%
14	src\assets\images\avatars\avatar-2.png	41.16 KB	0.13%
15	src\assets\images\avatars\avatar-3.png	41.06 KB	0.13%
16	src\assets\images\avatars\avatar-8.png	35.8 KB	0.11%
17	src\assets\images\avatars\avatar-5.png	31.46 KB	0.10%
18	src\ui\sidebar.tsx	21.85 KB	0.07%
19	src\components\upload\upload-illustration.tsx	21.84 KB	0.07%
20	src\pages\dashboard\analysis\index.tsx	21.06 KB	0.07%

此类问题，说明图片资源过大，需要将图片上CDN

6.2 判断未使用依赖

```
Code block

1  #!/usr/bin/env node
2
3  const fs = require('fs');
4  const path = require('path');
5
6  /**
7   * 读取package.json文件
8   * @param {string} projectRoot - 项目根目录
9   * @returns {object} package.json内容
10  */
11 function readPackageJson(projectRoot) {
12   const packageJsonPath = path.join(projectRoot, 'package.json');
13   try {
14     const content = fs.readFileSync(packageJsonPath, 'utf-8');
15     return JSON.parse(content);
16   } catch (error) {
17     console.error('读取package.json失败:', error.message);
18     process.exit(1);
19   }
20 }
21
22 /**
23  * 获取配置文件中的依赖引用
24  * @param {string} projectRoot - 项目根目录
25  * @returns {Set<string>} 配置文件中引用的依赖名称集合
26  */
27 function getDependenciesFromConfigs(projectRoot) {
28   const configs = new Set();
29   const configFiles = [
```

```

30     'vite.config.ts', 'vite.config.js',
31     'tailwind.config.ts', 'tailwind.config.js',
32     'tsconfig.json',
33     'postcss.config.js',
34     'babel.config.js',
35     '.eslintrc.js', '.eslintrc.cjs',
36     '.prettierrc.js',
37     'biome.json'
38 ];
39
40 configFiles.forEach(fileName => {
41     const filePath = path.join(projectRoot, fileName);
42     if (fs.existsSync(filePath)) {
43         try {
44             const content = fs.readFileSync(filePath, 'utf-8');
45             // 简单匹配配置文件中提到的包名
46             // 特别是tsconfig.json中的typeRoots和types
47             if (fileName === 'tsconfig.json') {
48                 const typesMatch = content.match(/"types":\s*\\[([^\]]+)\]\s*,?\|"typeRoots":\s*\[([^\]]+)\]\s*,?/);
49                 if (typesMatch) {
50                     const typesContent = typesMatch[1] || typesMatch[2];
51                     const types = typesContent.match(/"([^\"]+)"\/g) || [];
52                     types.forEach(type => {
53                         // 清理引号并添加到配置依赖中
54                         configs.add(type.replace(/"/g, ''));
55                     });
56                 }
57             }
58
59             // 匹配配置文件中的插件和预设
60             const pluginMatch = content.match(/"([^\"]+)"\s*@\w[\w\.-]*\w"\/g);
61             if (pluginMatch) {
62                 pluginMatch.forEach(match => {
63                     const dep = match.replace(/"\/g, '');
64                     // 过滤掉一些明显不是依赖的字符串
65                     if (dep && !dep.includes('/') && !dep.startsWith('./') && !dep.startsWith('../')) {
66                         configs.add(dep);
67                     }
68                 });
69             }
70         } catch (error) {
71             console.warn(`读取配置文件失败 ${filePath}: ${error.message}`);
72         }
73     }
74 });
75
76 return configs;
77 }
78
79 /**
80  * 扫描目录中的文件
81  * @param {string} dir - 要扫描的目录
82  * @param {array} extensions - 要扫描的文件扩展名列表
83  * @returns {array} 文件路径列表
84  */
85 function scanDirectory(dir, extensions) {
86     const files = [];
87
88     function scan(currentDir) {
89         try {
90             const entries = fs.readdirSync(currentDir, { withFileTypes: true });

```

```
91
92     for (const entry of entries) {
93         const fullPath = path.join(currentDir, entry.name);
94
95         if (entry.isDirectory()) {
96             // 跳过node_modules和其他不需要扫描的目录
97             if (entry.name === 'node_modules' || entry.name === '.git' || entry.name === 'dist' ||
entry.name === 'build') {
98                 continue;
99             }
100             scan(fullPath);
101         } else if (entry.isFile()) {
102             const ext = path.extname(fullPath).toLowerCase();
103             if (extensions.includes(ext)) {
104                 files.push(fullPath);
105             }
106         }
107     }
108 } catch (error) {
109     console.warn(`扫描目录失败 ${currentDir}: ${error.message}`);
110 }
111 }
112
113 scan(dir);
114 return files;
115 }
116
117 /**
118  * 从文件内容中提取导入的依赖
119  * @param {string} filePath - 文件路径
120  * @returns {Set<string>} 导入的依赖名称集合
121  */
122 function extractImports(filePath) {
123     const imports = new Set();
124     try {
125         const content = fs.readFileSync(filePath, 'utf-8');
126
127         // 匹配ESM导入语句（更全面的模式）
128         const esmImportRegex = /import\s+(?:type\s+)?(?:\s*\{\s*[\w$]+\s*\}\s*|\s*\*\s+as\s+\w+)\s+from\s+(['"])([^\s"]+)(['"])/g;
129         let match;
130         while ((match = esmImportRegex.exec(content)) !== null) {
131             const importPath = match[1];
132             // 提取包名（排除相对路径导入）
133             if (!importPath.startsWith('./') && !importPath.startsWith('../') && !importPath.startsWith('/')) {
134                 const packageName = importPath.split('/')[0];
135                 imports.add(packageName);
136             }
137         }
138
139         // 匹配CommonJS require语句
140         const cjsRequireRegex = /require\s*\(\s*['"]([^\s"]+)[\s"]\s*\)/g;
141         while ((match = cjsRequireRegex.exec(content)) !== null) {
142             const requirePath = match[1];
143             // 提取包名（排除相对路径导入）
144             if (!requirePath.startsWith('./') && !requirePath.startsWith('../') &&
!requirePath.startsWith('/')) {
145                 const packageName = requirePath.split('/')[0];
146                 imports.add(packageName);
147             }
148         }
149     }
```

```
150 // 匹配import.meta.glob语句
151 const globImportRegex = /import\.meta\.glob\(\s*['"](?:^'|')+['"]\s*\)/g;
152 while ((match = globImportRegex.exec(content)) !== null) {
153     const globPath = match[1];
154     // 处理可能包含依赖的glob导入
155     if (!globPath.startsWith('./') && !globPath.startsWith('../') && !globPath.startsWith('/')) {
156         const packageName = globPath.split('/')[0];
157         imports.add(packageName);
158     }
159 }
160
161 // 处理动态导入
162 const dynamicImportRegex = /import\(\s*['"](?:^'|')+['"]\s*\s*\)/g;
163 while ((match = dynamicImportRegex.exec(content)) !== null) {
164     const importPath = match[1];
165     if (!importPath.startsWith('./') && !importPath.startsWith('../') && !importPath.startsWith('/')) {
166         const packageName = importPath.split('/')[0];
167         imports.add(packageName);
168     }
169 }
170 } catch (error) {
171     console.warn(`读取文件失败 ${filePath}: ${error.message}`);
172 }
173
174 return imports;
175 }
176
177 /**
178  * 检查是否为可能在运行时使用的依赖
179  * @param {string} depName - 依赖名称
180  * @returns {boolean} 是否可能在运行时使用
181  */
182 function isRuntimeDependency(depName) {
183     const runtimePatterns = [
184         // TypeScript相关
185         /^@types\/,
186         // 构建工具
187         /^@vitejs\/,
188         /^@babel\/,
189         /^@biomejs\/,
190         /^tailwindcss$/,
191         /^@tailwindcss\/,
192         /^postcss$/,
193         /^eslint$/,
194         /^@eslint\/,
195         /^prettier$/,
196         /^typescript$/,
197         /^@tanstack\/,
198         /^@hookform\/,
199         /^@fullcalendar\/,
200         /^@iconify\/,
201         /^lefthook$/,
202         /^@commitlint\/,
203         /^@vanilla-extract\/,
204         /^@fontsource/,
205         /^@faker-js\/,
206         /^react-scan$/,
207         /^babel-plugin-/ ,
208         // 常用开发工具
209         /^tw-/ , // tailwind插件
210         /^date-fns$/,
211         /^@vercel\/
```

```
212     ];
213     return runtimePatterns.some(pattern => pattern.test(depName));
214 }
215
216 /**
217  * 分析未使用的依赖
218  */
219 function analyzeUnusedDependencies() {
220     const projectRoot = path.join(__dirname, '..');
221     const packageJson = readPackageJson(projectRoot);
222
223     // 合并dependencies和devDependencies
224     const allDependencies = {
225         ...packageJson.dependencies || {},
226         ...packageJson.devDependencies || {}
227     };
228
229     console.log('正在扫描项目文件...');
230
231     // 扫描源代码文件（增加更多文件类型）
232     const sourceFiles = scanDirectory(projectRoot, ['.js', '.jsx', '.ts', '.tsx', '.mjs', '.cjs', '.json',
233     '.css', '.scss', '.sass', '.less', '.html', '.vue']);
234
235     console.log(`找到 ${sourceFiles.length} 个源代码文件`);
236     console.log('正在分析导入的依赖...');
237
238     // 收集所有使用的依赖
239     const usedDependencies = new Set();
240
241     sourceFiles.forEach((file, index) => {
242         // 显示进度
243         if ((index + 1) % 50 === 0 || index + 1 === sourceFiles.length) {
244             process.stdout.write(`\r已处理 ${index + 1}/${sourceFiles.length} 个文件`);
245         }
246
247         const imports = extractImports(file);
248         imports.forEach(imp => usedDependencies.add(imp));
249     });
250
251     // 从配置文件中获取依赖引用
252     console.log('\n正在分析配置文件中的依赖引用...');
253     const configDependencies = getDependenciesFromConfigs(projectRoot);
254     configDependencies.forEach(dep => usedDependencies.add(dep));
255
256     // 处理一些特殊情况：检查package.json中的scripts是否引用了某些依赖
257     console.log('正在分析package.json中的scripts引用...');
258     if (packageJson.scripts) {
259         Object.values(packageJson.scripts).forEach(script => {
260             // 简单匹配脚本中可能使用的全局命令，这些通常对应于依赖
261             const scriptDeps = script.match(/(?:b(?:npx\s+)?(?:@?[\w-]+\s+)?b/g) || [];
262             scriptDeps.forEach(scriptDep => {
263                 // 移除npx前缀
264                 const cleanDep = scriptDep.replace(/^npx\s+/, '');
265                 // 如果这是package.json中的依赖，则标记为使用
266                 if (allDependencies[cleanDep]) {
267                     usedDependencies.add(cleanDep);
268                 }
269             });
270         });
271     }
272
273     console.log('\n\n=== 未使用的依赖分析结果 ===');
```

```
273 console.log('注意：此分析可能存在误报，特别是对于在特定条件下或通过特殊方式使用的依赖。');
274
275 // 分类依赖
276 const unusedDependencies = {};
277 const usedDependenciesList = {};
278 const possibleRuntimeDeps = {};
279
280 // 检查dependencies中的未使用依赖
281 if (packageJson.dependencies) {
282   Object.keys(packageJson.dependencies).forEach(dep => {
283     if (!usedDependencies.has(dep)) {
284       // 对于可能在运行时使用的依赖，标记为可能的误报
285       if (isRuntimeDependency(dep)) {
286         possibleRuntimeDeps[dep] = packageJson.dependencies[dep];
287       } else {
288         unusedDependencies[dep] = packageJson.dependencies[dep];
289       }
290     } else {
291       usedDependenciesList[dep] = packageJson.dependencies[dep];
292     }
293   });
294 }
295
296 // 检查devDependencies中的未使用依赖
297 if (packageJson.devDependencies) {
298   Object.keys(packageJson.devDependencies).forEach(dep => {
299     if (!usedDependencies.has(dep)) {
300       // 对于可能在运行时使用的依赖，标记为可能的误报
301       if (isRuntimeDependency(dep)) {
302         possibleRuntimeDeps[dep] = packageJson.devDependencies[dep];
303       } else {
304         unusedDependencies[dep] = packageJson.devDependencies[dep];
305       }
306     } else {
307       usedDependenciesList[dep] = packageJson.devDependencies[dep];
308     }
309   });
310 }
311
312 // 输出使用的依赖
313 console.log(`\n已使用的依赖 (${Object.keys(usedDependenciesList).length}):`);
314 Object.keys(usedDependenciesList).sort().forEach(dep => {
315   console.log(`  ${dep}: ${usedDependenciesList[dep]}`);
316 });
317
318 // 输出可能误报的依赖
319 if (Object.keys(possibleRuntimeDeps).length > 0) {
320   console.log(`\n可能被误报为未使用的依赖 (${Object.keys(possibleRuntimeDeps).length}):`);
321   console.log('  这些依赖可能在特殊情况下使用，建议谨慎处理：');
322   Object.keys(possibleRuntimeDeps).sort().forEach(dep => {
323     console.log(`  ${dep}: ${possibleRuntimeDeps[dep]}`);
324   });
325 }
326
327 // 输出未使用的依赖
328 console.log(`\n未使用的依赖 (${Object.keys(unusedDependencies).length}):`);
329 if (Object.keys(unusedDependencies).length === 0) {
330   console.log('  恭喜！没有发现明确未使用的依赖。');
331 } else {
332   Object.keys(unusedDependencies).sort().forEach(dep => {
333     console.log(`  ${dep}: ${unusedDependencies[dep]}`);
334   });
335 }
```

```

335
336     console.log('\n建议：您可以考虑使用以下命令删除未使用的依赖：');
337     console.log(`  npm uninstall ${Object.keys(unusedDependencies).join(' ')}`);
338   }
339
340   // 输出一些统计信息
341   const totalDeps = Object.keys(allDependencies).length;
342   const unusedCount = Object.keys(unusedDependencies).length;
343   const usagePercentage = totalDeps > 0 ? ((totalDeps - unusedCount -
Object.keys(possibleRuntimeDeps).length) / totalDeps * 100).toFixed(1) : 0;
344
345   console.log(`\n=== 统计信息 ===`);
346   console.log(`总依赖数：${totalDeps}`);
347   console.log(`明确使用中的依赖数：${Object.keys(usedDependenciesList).length}`);
348   console.log(`可能被误报的依赖数：${Object.keys(possibleRuntimeDeps).length}`);
349   console.log(`明确未使用的依赖数：${unusedCount}`);
350   console.log(`保守估计的依赖使用率：${usagePercentage}%`);
351 }
352
353 // 执行分析
354 analyzeUnusedDependencies();

```

6.3 分析重复代码

Code block

```

1  #!/usr/bin/env node
2
3  const { execSync } = require('child_process');
4  const fs = require('fs');
5  const path = require('path');
6  const readline = require('readline');
7
8  const colors = {
9    reset: '\x1b[0m',
10   green: '\x1b[32m',
11   yellow: '\x1b[33m',
12   blue: '\x1b[34m',
13   magenta: '\x1b[35m',
14   cyan: '\x1b[36m',
15   red: '\x1b[31m',
16   white: '\x1b[37m'
17 };
18
19 const projectRoot = path.resolve(process.cwd(), './');
20
21 const outputDir = path.join(projectRoot, 'analysis-duplication');
22
23 if (!fs.existsSync(outputDir)) {
24   fs.mkdirSync(outputDir, { recursive: true });
25 }
26
27 const defaultParams = {
28   srcDir: path.join(projectRoot, 'src'),
29   threshold: 10, // 重复度阈值 (0-100)
30   minLines: 8, // 最小重复行数
31   minTokens: 70, // 最小重复tokens数
32   includePatterns: '**/*. {tsx,ts,jsx,js,d.ts,scss}', // 包含的文件格式

```

```
33   excludePatterns: ['**/*.spec.ts', '**/*.test.ts', '**/*.spec.tsx', '**/*.test.tsx',
34   '**/node_modules/**', '**/dist/**'], // 排除的文件格式
35 };
36
37 const rl = readline.createInterface({
38   input: process.stdin,
39   output: process.stdout
40 });
41
42 function askQuestion(query) {
43   return new Promise(resolve => rl.question(query, resolve));
44 }
45
46 async function configureAnalysisParams() {
47   console.log(`${colors.cyan}===== ${colors.reset}`);
48   console.log(`${colors.cyan}代码重复度分析配置 ${colors.reset}`);
49   console.log(`${colors.cyan}===== ${colors.reset}`);
50   console.log(`${colors.yellow}请配置分析参数 (直接回车使用默认值): ${colors.reset}`);
51
52   const analysisParams = { ...defaultParams };
53
54   // 询问源代码目录
55   const srcDirInput = await askQuestion(`${colors.blue}\n源代码目录 [${path.relative(projectRoot,
56   analysisParams.srcDir)}]: ${colors.reset}`);
57   if (srcDirInput.trim()) {
58     analysisParams.srcDir = path.isAbsolute(srcDirInput) ? srcDirInput : path.join(projectRoot,
59   srcDirInput);
60   }
61
62   // 询问重复度阈值
63   const thresholdInput = await askQuestion(`${colors.blue}重复度阈值 (%) [${analysisParams.threshold}]:
64   ${colors.reset}`);
65   if (thresholdInput.trim()) {
66     const threshold = parseInt(thresholdInput, 10);
67     if (!isNaN(threshold) && threshold >= 0 && threshold <= 100) {
68       analysisParams.threshold = threshold;
69     }
70   }
71
72   // 询问最小重复行数
73   const minLinesInput = await askQuestion(`${colors.blue}最小重复行数 [${analysisParams.minLines}]:
74   ${colors.reset}`);
75   if (minLinesInput.trim()) {
76     const minLines = parseInt(minLinesInput, 10);
77     if (!isNaN(minLines) && minLines > 0) {
78       analysisParams.minLines = minLines;
79     }
80   }
81
82   // 询问最小重复tokens数
83   const minTokensInput = await askQuestion(`${colors.blue}最小重复tokens数 [${analysisParams.minTokens}]:
84   ${colors.reset}`);
85   if (minTokensInput.trim()) {
86     const minTokens = parseInt(minTokensInput, 10);
87     if (!isNaN(minTokens) && minTokens > 0) {
88       analysisParams.minTokens = minTokens;
89     }
90   }
91
92   // 询问包含模式
```

```
88     const includePatternsInput = await askQuestion(`${colors.blue}包含文件模式
[${analysisParams.includePatterns}]: ${colors.reset}`);
89     if (includePatternsInput.trim()) {
90         analysisParams.includePatterns = includePatternsInput.trim();
91     }
92
93     // 询问排除模式
94     const excludePatternsInput = await askQuestion(`${colors.blue}排除文件格式（用逗号分隔）
[${analysisParams.excludePatterns.join(', ')}]: ${colors.reset}`);
95     if (excludePatternsInput.trim()) {
96         analysisParams.excludePatterns = excludePatternsInput.split(',').map(pattern => pattern.trim());
97     }
98
99     // 询问导出格式
100    const outputFormatsInput = await askQuestion(`${colors.blue}导出格式（用逗号分隔）
[${analysisParams.outputFormats.join(', ')}]: ${colors.reset}`);
101    if (outputFormatsInput.trim()) {
102        analysisParams.outputFormats = outputFormatsInput.split(',').map(format => format.trim());
103    }
104
105    // 关闭readline接口
106    rl.close();
107
108    return analysisParams;
109 }
110
111 async function main() {
112     try {
113         const analysisParams = await configureAnalysisParams();
114
115         console.log(`${colors.green}\n===== ${colors.reset}`);
116         console.log(`${colors.green}开始代码重复度分析... ${colors.reset}`);
117         console.log(`${colors.green}===== ${colors.reset}\n`);
118
119         console.log(`${colors.yellow}分析配置: ${colors.reset}`);
120         console.log(`${colors.blue}- 源代码目录: ${colors.white}${analysisParams.srcDir}${colors.reset}`);
121         console.log(`${colors.blue}- 重复度阈值: ${colors.white}${analysisParams.threshold}% ${colors.reset}`);
122         console.log(`${colors.blue}- 最小重复行数: ${colors.white}${analysisParams.minLines}${colors.reset}`);
123         console.log(`${colors.blue}- 最小重复tokens数:
${colors.white}${analysisParams.minTokens}${colors.reset}`);
124         console.log(`${colors.blue}- 包含模式:
${colors.white}${analysisParams.includePatterns}${colors.reset}`);
125         console.log(`${colors.blue}- 排除模式: ${colors.white}${analysisParams.excludePatterns.join(',
')} ${colors.reset}`);
126         console.log(`${colors.blue}- 输出格式: ${colors.white}${analysisParams.outputFormats.join(',
')} ${colors.reset}`);
127         console.log(`${colors.yellow}\n分析过程中, 请稍候... ${colors.reset}\n`);
128
129         const excludeArgs = analysisParams.excludePatterns.map(pattern => `--ignore "${pattern}"`).join(' ');
130
131         const jscpdCommand = `npx jscpd "${analysisParams.srcDir}" --pattern
"${analysisParams.includePatterns}" --threshold ${analysisParams.threshold} --min-lines
${analysisParams.minLines} --min-tokens ${analysisParams.minTokens} ${excludeArgs} --reporters
"html","json","console" --output "${outputDir}" --verbose`;
132
133         execSync(jscpdCommand, { encoding: 'utf-8', stdio: 'inherit' });
134
135         const jsonReportPath = path.join(outputDir, 'jscpd-report.json');
136         if (fs.existsSync(jsonReportPath)) {
137             const report = JSON.parse(fs.readFileSync(jsonReportPath, 'utf-8'));
138
139             console.log(`${colors.cyan}\n===== ${colors.reset}`);
```

```
140     console.log(`${colors.cyan}代码重复度分析结果摘要${colors.reset}`);
141     console.log(`${colors.cyan}=====${colors.reset}`);
142     console.log(`${colors.blue}- 总文件数:
${colors.white}${report.statistics.total.sources}${colors.reset}`);
143     console.log(`${colors.blue}- 总行数:
${colors.white}${report.statistics.total.lines}${colors.reset}`);
144     console.log(`${colors.blue}- 总tokens数:
${colors.white}${report.statistics.total.tokens}${colors.reset}`);
145     console.log(`${colors.blue}- 重复文件数:
${colors.white}${report.duplicates.length}${colors.reset}`);
146     console.log(`${colors.blue}- 重复行数:
${colors.white}${report.statistics.total.duplicatedLines}${colors.reset}`);
147     console.log(`${colors.blue}- 重复tokens数:
${colors.white}${report.statistics.total.duplicatedTokens}${colors.reset}`);
148     console.log(`${colors.blue}- 整体重复度:
${colors.white}${(report.statistics.total.percentage).toFixed(2)}%${colors.reset}`);
149
150     if (report.duplicates && report.duplicates.length > 0) {
151         console.log(`${colors.magenta}\n=====${colors.reset}`);
152         console.log(`${colors.magenta}最严重的重复代码片段 (前5个)${colors.reset}`);
153         console.log(`${colors.magenta}=====${colors.reset}`);
154
155         const topDuplicates = report.duplicates
156             .sort((a, b) => b.lines - a.lines) // 按重复行数排序
157             .slice(0, 5);
158
159         topDuplicates.forEach((dup, index) => {
160             try {
161                 // 计算相似度
162                 let similarity = '';
163                 if (dup.lines && (dup.end !== undefined && dup.start !== undefined)) {
164                     similarity = `, 相似度: ${colors.red}${((dup.lines / (dup.end - dup.start + 1)) *
100).toFixed(1)}%${colors.yellow}`;
165                 }
166
167                 // 显示重复行数和相似度
168                 console.log(`${colors.yellow}\n${index + 1}. 重复行数:
${colors.white}${dup.lines}${similarity}${colors.reset}`);
169
170                 // 显示文件格式信息
171                 if (dup.format) {
172                     console.log(`${colors.blue} 文件格式: ${colors.white}${dup.format}${colors.reset}`);
173                 }
174
175                 // 处理jscpd报告的主要结构 - firstFile和secondFile对象
176                 if (dup.firstFile && dup.secondFile) {
177                     // 处理第一个文件信息
178                     const firstFile = typeof dup.firstFile === 'string' ? { name: dup.firstFile } :
dup.firstFile;
179                     const firstFileName = firstFile.name || '';
180                     const firstFilePath = path.relative(projectRoot, firstFileName);
181                     const firstFileStart = firstFile.start !== undefined ? firstFile.start :
182                         (dup.firstFileStart !== undefined ? dup.firstFileStart : 0);
183                     const firstFileEnd = firstFile.end !== undefined ? firstFile.end :
184                         (dup.firstFileEnd !== undefined ? dup.firstFileEnd : 0);
185
186                     // 处理第二个文件信息
187                     const secondFile = typeof dup.secondFile === 'string' ? { name: dup.secondFile } :
dup.secondFile;
188                     const secondFileName = secondFile.name || '';
189                     const secondFilePath = path.relative(projectRoot, secondFileName);
190                     const secondFileStart = secondFile.start !== undefined ? secondFile.start :
```

```
191         (dup.secondFileStart !== undefined ? dup.secondFileStart : 0);
192     const secondFileEnd = secondFile.end !== undefined ? secondFile.end :
193         (dup.secondFileEnd !== undefined ? dup.secondFileEnd : 0);
194
195     // 显示文件路径和行号信息
196     console.log(`${colors.green}    文件1: ${colors.white}${firstFilePath}${colors.reset}`);
197     if (firstFileStart && firstFileEnd) {
198         console.log(`${colors.green}    文件1行号:
199     ${colors.white}${firstFileStart}${colors.green} - ${colors.white}${firstFileEnd}${colors.reset}`);
200     }
201
202     console.log(`${colors.green}    文件2: ${colors.white}${secondFilePath}${colors.reset}`);
203     if (secondFileStart && secondFileEnd) {
204         console.log(`${colors.green}    文件2行号:
205     ${colors.white}${secondFileStart}${colors.green} - ${colors.white}${secondFileEnd}${colors.reset}`);
206     }
207     } else if (dup.files && Array.isArray(dup.files) && dup.files.length >= 2) {
208         // 处理files数组结构（兼容旧版jscpd报告）
209         dup.files.forEach((file, fileIndex) => {
210             if (file && (file.name || typeof file === 'string')) {
211                 const fileName = file.name || file;
212                 const filePath = path.relative(projectRoot, fileName);
213                 console.log(`${colors.green}    文件${fileIndex + 1}:
214     ${colors.white}${filePath}${colors.reset}`);
215
216                 if (file.start !== undefined && file.end !== undefined) {
217                     console.log(`${colors.green}    文件${fileIndex + 1}行号:
218     ${colors.white}${file.start}${colors.green} - ${colors.white}${file.end}${colors.reset}`);
219                 }
220             }
221         });
222     } else {
223         // 显示可用的字段信息，便于调试
224         console.log(`${colors.yellow}    文件信息结构:${colors.reset}`);
225         console.log(`${colors.yellow}    可用字段: ${Object.keys(dup).join(', ')}${colors.reset}`);
226
227         // 尝试提取任何可能的文件信息
228         if (dup.source && typeof dup.source === 'object') {
229             const sourceFile = dup.source.file || (typeof dup.source === 'string' ? dup.source :
230     JSON.stringify(dup.source).substring(0, 50));
231             if (sourceFile) {
232                 console.log(`${colors.yellow}    source:
233     ${colors.white}${sourceFile}${colors.reset}`);
234             }
235         }
236
237         if (dup.target && typeof dup.target === 'object') {
238             const targetFile = dup.target.file || (typeof dup.target === 'string' ? dup.target :
239     JSON.stringify(dup.target).substring(0, 50));
240             if (targetFile) {
241                 console.log(`${colors.yellow}    target:
242     ${colors.white}${targetFile}${colors.reset}`);
243             }
244         }
245     } catch (err) {
246         // 错误处理，确保即使出现异常也能继续执行
247         console.log(`${colors.red}\n${index + 1}. 处理重复代码片段时出错:${colors.reset}`);
248         console.log(`${colors.red}    错误信息: ${err.message}${colors.reset}`);
249
250         // 尝试显示基本信息
251         if (dup) {
252             console.log(`${colors.red}    文件: ${dup.filePath}${colors.reset}`);
253             console.log(`${colors.red}    行号: ${dup.start} - ${dup.end}${colors.reset}`);
254             console.log(`${colors.red}    内容: ${JSON.stringify(dup.content).substring(0, 100)}${colors.reset}`);
255         }
256     }
257 }
```

```

245         console.log(`${colors.yellow}    重复行数: ${colors.white}${dup.lines} || '未知'`${colors.reset}`);
246         console.log(`${colors.yellow}    片段ID: ${colors.white}${index + 1}${colors.reset}`);
247     }
248 }
249 });
250 }
251
252 const htmlReportPath = path.join(outputDir, 'html/index.html');
253 if (fs.existsSync(htmlReportPath)) {
254     console.log(`${colors.cyan}\n=====${colors.reset}`);
255     console.log(`${colors.cyan}详细报告${colors.reset}`);
256     console.log(`${colors.cyan}=====${colors.reset}`);
257     console.log(`${colors.blue}- 查看HTML报告: ${colors.green}${htmlReportPath}${colors.reset}`);
258     console.log(`${colors.blue}- 查看JSON报告: ${colors.green}${jsonReportPath}${colors.reset}`);
259 }
260 }
261
262 console.log(`${colors.green}\n=====${colors.reset}`);
263 console.log(`${colors.green}代码重复度分析完成${colors.reset}`);
264 console.log(`${colors.green}=====${colors.reset}\n`);
265
266 } catch (error) {
267     console.error(`${colors.red}分析过程中发生错误:${colors.white} ${error.message}${colors.reset}`);
268     process.exit(1);
269 }
270 }
271
272 main();

```

6.4 判断无引用文件

Code block

```

1  #!/usr/bin/env node
2
3  const fs = require('fs');
4  const path = require('path');
5  const { promisify } = require('util');
6
7  // 颜色输出工具
8  const colors = {
9      reset: '\x1b[0m',
10     green: '\x1b[32m',
11     yellow: '\x1b[33m',
12     blue: '\x1b[34m',
13     magenta: '\x1b[35m',
14     cyan: '\x1b[36m',
15     red: '\x1b[31m',
16     white: '\x1b[37m'
17 };
18
19 // 项目根目录
20 const projectRoot = path.resolve(process.cwd(), './');
21 const srcDir = path.join(projectRoot, 'src');
22
23 // 要分析的文件扩展名
24 const VALID_EXTENSIONS = ['.js', '.jsx', '.ts', '.tsx', '.vue', '.css', '.scss', '.less'];
25
26 // 忽略的目录

```

```
27  const IGNORE_DIRS = [
28    'node_modules',
29    'dist',
30    'build',
31    '.git',
32    '.vscode',
33    '.cursor',
34    'public',
35    'analysis-duplication',
36    '_mock'
37  ];
38
39  // 忽略的文件
40  const IGNORE_FILES = [
41    'vite-env.d.ts',
42    'main.tsx', // 入口文件通常不会被引用
43    'index.html',
44    'package.json',
45    'tsconfig.json',
46    'tailwind.config.ts',
47    'vite.config.ts',
48    'App.tsx' // 应用主组件
49  ];
50
51  // 异步读取文件
52  const readFile = promisify(fs.readFile);
53
54  /**
55   * 递归获取目录下的所有文件
56   */
57  async function getAllFiles(dir) {
58    let files = [];
59
60    const entries = await fs.promises.readdir(dir, { withFileTypes: true });
61
62    for (const entry of entries) {
63      const fullPath = path.join(dir, entry.name);
64
65      // 忽略指定目录
66      if (entry.isDirectory() && IGNORE_DIRS.includes(entry.name)) {
67        continue;
68      }
69
70      if (entry.isDirectory()) {
71        const subFiles = await getAllFiles(fullPath);
72        files = [...files, ...subFiles];
73      } else if (entry.isFile()) {
74        // 只处理有效的文件扩展名
75        const ext = path.extname(entry.name);
76        if (VALID_EXTENSIONS.includes(ext) && !IGNORE_FILES.includes(entry.name)) {
77          files.push(fullPath);
78        }
79      }
80    }
81
82    return files;
83  }
84
85  /**
86   * 从文件路径获取模块名（用于匹配import语句）
87   */
88  function getModuleName(filePath, projectRoot) {
```

```
89     const relativePath = path.relative(projectRoot, filePath);
90     // 移除扩展名
91     const nameWithoutExt = relativePath.replace(/\.[^/.]+$/, '');
92     // 转换为可导入的路径格式
93     return nameWithoutExt.replace(/\\/g, '/');
94 }
95
96 /**
97  * 解析文件中的导入语句
98  */
99 function extractImports(content, filePath, projectRoot) {
100     const imports = [];
101     const relativePath = path.relative(projectRoot, filePath);
102     const dirPath = path.dirname(relativePath).replace(/\\/g, '/');
103
104     // 匹配ES模块导入
105     const esImportRegex = /import\s+(?:.*?)\s+from\s+['"](.+?)['"]/g;
106     let match;
107     while ((match = esImportRegex.exec(content)) !== null) {
108         imports.push(match[1]);
109     }
110
111     // 匹配require导入
112     const requireRegex = /require\(['"](.+?)['"]\)/g;
113     while ((match = requireRegex.exec(content)) !== null) {
114         imports.push(match[1]);
115     }
116
117     // 匹配动态导入
118     const dynamicImportRegex = /import\(['"](.+?)['"]\)/g;
119     while ((match = dynamicImportRegex.exec(content)) !== null) {
120         imports.push(match[1]);
121     }
122
123     return imports;
124 }
125
126 /**
127  * 检查文件是否被引用
128  */
129 function isFileReferenced(filePath, allImports, projectRoot) {
130     const relativePath = path.relative(projectRoot, filePath);
131     const moduleName = getModuleName(filePath, projectRoot);
132     const fileName = path.basename(filePath);
133     const fileNameWithoutExt = fileName.replace(/\.[^/.]+$/, '');
134     const dirName = path.basename(path.dirname(filePath));
135
136     // 特殊处理组件和UI相关文件，它们可能通过配置或路由表引用
137     if (relativePath.includes('/components/') ||
138         relativePath.includes('/ui/') ||
139         relativePath.includes('/pages/') ||
140         relativePath.includes('/routes/')) {
141         // 这些文件可能通过路由配置、组件库配置等方式引用
142         // 检查是否有导入包含文件名或组件名
143         const componentLikeImports = allImports.some(importPath => {
144             // 检查是否包含文件名（可能是组件名）
145             return importPath.includes(fileNameWithoutExt) ||
146                 importPath.includes(camelCaseToKebab(fileNameWithoutExt)) ||
147                 importPath.includes(PascalCaseToKebab(fileNameWithoutExt));
148         });
149         if (componentLikeImports) {
150             return true;
```

```
151     }
152 }
153
154 // 检查是否有导入路径匹配此文件
155 return allImports.some(importPath => {
156     // 检查完整路径匹配
157     if (importPath === moduleName ||
158         importPath === moduleName + '.js' ||
159         importPath === moduleName + '.ts' ||
160         importPath === moduleName + '.jsx' ||
161         importPath === moduleName + '.tsx' ||
162         importPath === moduleName + '/index' ||
163         importPath === moduleName + '/index.js' ||
164         importPath === moduleName + '/index.ts' ||
165         importPath === moduleName + '/index.jsx' ||
166         importPath === moduleName + '/index.tsx') {
167         return true;
168     }
169
170     // 检查相对路径匹配
171     const possibleRelativePaths = [
172         './' + fileName,
173         './' + fileNameWithoutExt,
174         '../' + fileName,
175         '../' + fileNameWithoutExt,
176         './' + fileNameWithoutExt + '/index',
177         '../' + fileNameWithoutExt + '/index',
178         './' + dirName + '/' + fileNameWithoutExt,
179         '../' + dirName + '/' + fileNameWithoutExt
180     ];
181
182     return possibleRelativePaths.some(relPath => {
183         // 精确匹配相对路径，避免部分匹配导致的误报
184         return importPath === relPath ||
185             importPath.startsWith(relPath + '/') ||
186             importPath === relPath + '.js' ||
187             importPath === relPath + '.ts' ||
188             importPath === relPath + '.jsx' ||
189             importPath === relPath + '.tsx';
190     });
191 });
192 }
193
194 /**
195  * 驼峰命名转短横线命名
196  */
197 function camelCaseToKebab(str) {
198     return str.replace(/[A-Z]/g, letter => `-${letter.toLowerCase()}`);
199 }
200
201 /**
202  * 帕斯卡命名转短横线命名
203  */
204 function PascalCaseToKebab(str) {
205     return str.charAt(0).toLowerCase() + str.slice(1).replace(/[A-Z]/g, letter =>
206     `-${letter.toLowerCase()}`);
207 }
208
209 /**
210  * 主函数
211  */
211 async function main() {
```

```
212 try {
213   console.log(`${colors.cyan}===== ${colors.reset}`);
214   console.log(`${colors.cyan}未引用文件分析工具 ${colors.reset}`);
215   console.log(`${colors.cyan}===== ${colors.reset}`);
216
217   console.log(`${colors.yellow}开始扫描项目文件... ${colors.reset}`);
218   const allFiles = await getAllFiles(srcDir);
219   console.log(`${colors.green}共扫描到 ${allFiles.length} 个文件 ${colors.reset}`);
220
221   console.log(`${colors.yellow}\n开始分析文件引用关系... ${colors.reset}`);
222   const allImports = [];
223   const fileContents = new Map();
224
225   // 读取所有文件内容并提取导入
226   for (let i = 0; i < allFiles.length; i++) {
227     const filePath = allFiles[i];
228     try {
229       const content = await readFile(filePath, 'utf-8');
230       fileContents.set(filePath, content);
231       const imports = extractImports(content, filePath, projectRoot);
232       allImports.push(...imports);
233
234       // 显示进度
235       if ((i + 1) % 10 === 0 || i === allFiles.length - 1) {
236         console.log(`${colors.blue}进度: ${i + 1}/${allFiles.length} 个文件 ${colors.reset}`);
237       }
238     } catch (error) {
239       console.log(`${colors.red}读取文件失败: ${filePath} - ${error.message} ${colors.reset}`);
240     }
241   }
242
243   console.log(`${colors.yellow}\n开始识别未引用文件... ${colors.reset}`);
244   const unusedFiles = [];
245
246   for (const filePath of allFiles) {
247     if (!isFileReferenced(filePath, allImports, projectRoot)) {
248       unusedFiles.push(filePath);
249     }
250   }
251
252   console.log(`${colors.cyan}\n===== ${colors.reset}`);
253   console.log(`${colors.cyan}分析结果 ${colors.reset}`);
254   console.log(`${colors.cyan}===== ${colors.reset}`);
255
256   console.log(`${colors.blue}- 总文件数: ${colors.white}${allFiles.length} ${colors.reset}`);
257   // 按目录分组未引用文件
258   const unusedFilesByDir = {};
259   unusedFiles.forEach(filePath => {
260     const dir = path.dirname(filePath);
261     const relativeDir = path.relative(projectRoot, dir);
262     if (!unusedFilesByDir[relativeDir]) {
263       unusedFilesByDir[relativeDir] = [];
264     }
265     unusedFilesByDir[relativeDir].push(filePath);
266   });
267
268   console.log(`${colors.blue}- 未引用文件数: ${colors.white}${unusedFiles.length} ${colors.reset}`);
269
270   if (unusedFiles.length > 0) {
271     console.log(`${colors.yellow}\n未引用的文件列表(按目录分组): ${colors.reset}`);
272
273     Object.entries(unusedFilesByDir).forEach(([dir, files]) => {
```

```

274     console.log(`${colors.cyan}\n${dir} (${files.length} 个文件):${colors.reset}`);
275     files.forEach((file, index) => {
276         const relativePath = path.relative(projectRoot, file);
277         console.log(`${colors.red}  ${index + 1}.  ${path.basename(file)}${colors.reset}`);
278     });
279 });
280
281 console.log(`${colors.yellow}\n未引用文件详情列表:${colors.reset}`);
282 unusedFiles.forEach((file, index) => {
283     const relativePath = path.relative(projectRoot, file);
284     console.log(`${colors.red}${index + 1}.  ${relativePath}${colors.reset}`);
285 });
286
287 // 计算未引用文件占用的磁盘空间
288 let totalSize = 0;
289 for (const filePath of unusedFiles) {
290     try {
291         const stats = fs.statSync(filePath);
292         totalSize += stats.size;
293     } catch (error) {
294         // 忽略错误
295     }
296 }
297
298 console.log(`${colors.yellow}\n未引用文件总计大小: ${colors.white}${(totalSize / 1024).toFixed(2)}
KB${colors.reset}`);
299
300 } else {
301     console.log(`${colors.green}\n恭喜! 没有发现未引用的文件。${colors.reset}`);
302 }
303
304 console.log(`${colors.cyan}\n=====${colors.reset}`);
305 console.log(`${colors.cyan}分析完成${colors.reset}`);
306 console.log(`${colors.cyan}=====${colors.reset}`);
307
308 } catch (error) {
309     console.error(`${colors.red}分析过程中发生错误:${colors.white}  ${error.message}${colors.reset}`);
310     console.error(error.stack);
311     process.exit(1);
312 }
313 }
314
315 // 执行主函数
316 main();

```

6.5 npm场景下使用 PeerDependencies

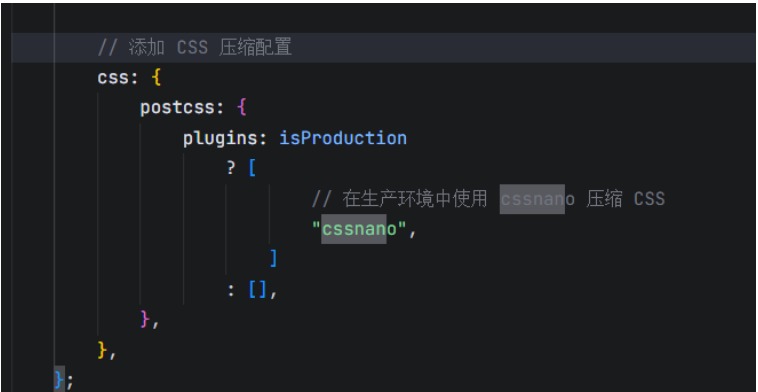
因此为实现所有依赖外部化，因此需要从 dependencies 中移除重复的依赖，并移动到 devDependencies 中。

6.6 替换相同功能，体积更小的依赖

- Moment -> dayjs
- Lodash -> lodash-es

6.7 Css 压缩

使用css nano



7. 基于 Vite 和 React Compiler 的 INP 优化

React Compiler 可以通过编译时优化减少运行时开销，提升应用性能。为了进一步优化用户体验，提高渲染性能，通过集成 React Compiler进行性能提升。

7.1 前置判断

通过 `npx react-compiler-healthcheck@latest` 进行前置校验。

Code block

- 1 Successfully compiled 640 out of 640 components.
- 2 StrictMode usage not found.
- 3 Found no usage of incompatible libraries

可以看到，存量代码还是有比较多可以运行时优化的空间。

7.2 功能集成

参考：<https://react.dev/learn/react-compiler>

技术/依赖	版本	用途
babel-plugin-react-compiler	^1.0.0	核心编译插件
react-compiler-runtime	^1.0.0	运行时支持，仅限于 react 19 以下
vite-plugin-babel	^1.3.2	Vite 与 Babel 的桥接
@babel/preset-react	^7.27.1	兼容 JSX 语法支持

通过 `reactCompilerPlugin` 函数封装对应配置：

Code block

- ```
1 import type { Plugin } from 'vite'
2 import babel from 'vite-plugin-babel'
3
4 export function reactCompilerPlugin(): Plugin {
5 const isDevelopment = process.env.NODE_ENV === 'development'
6
7 return babel({
8 babelConfig: {
9 presets: ['@babel/preset-react'],
10 plugins: [
11 [
12 'babel-plugin-react-compiler',
13 {
14 compilationMode: 'infer', // 自动推断编译模式
15 panicThreshold: isDevelopment ? 'critical_errors' : 'none', // 开发环境下仅对严重错误中断
```

```
16 target: '19', // 目标 React 版本
17 }
18]
19]
20 }
21 })
22 }
```

在构建产物脚本中，添加对应的插件配置。

Code block

```
1 export default defineConfig(() => {
2 return {
3 plugins: [
4 reactCompilerPlugin(),
5 // ... 其他插件
6];
7 });
8 }
```

## 7.3 效果验证

在本地添加配置后，通过 devTools 和 编译后产物 进行验证：

```
),Y=Symbol.for("react.memo_cache_sentinel"),au=
```